

UXRVT: Unity XR Visualization Toolkit for Large Datasets

Walid Chtioui Gourav Srinivasalu

- D3.js-document-like interface for creating highly customizable visualizations:

```
// compute the extent of relevant attributes
Utils.Extent(data, d => d.Date, out DateTime min_date, out DateTime max_date);
Utils.Extent(data, d => d.Value, out float min_val, out float max_val);

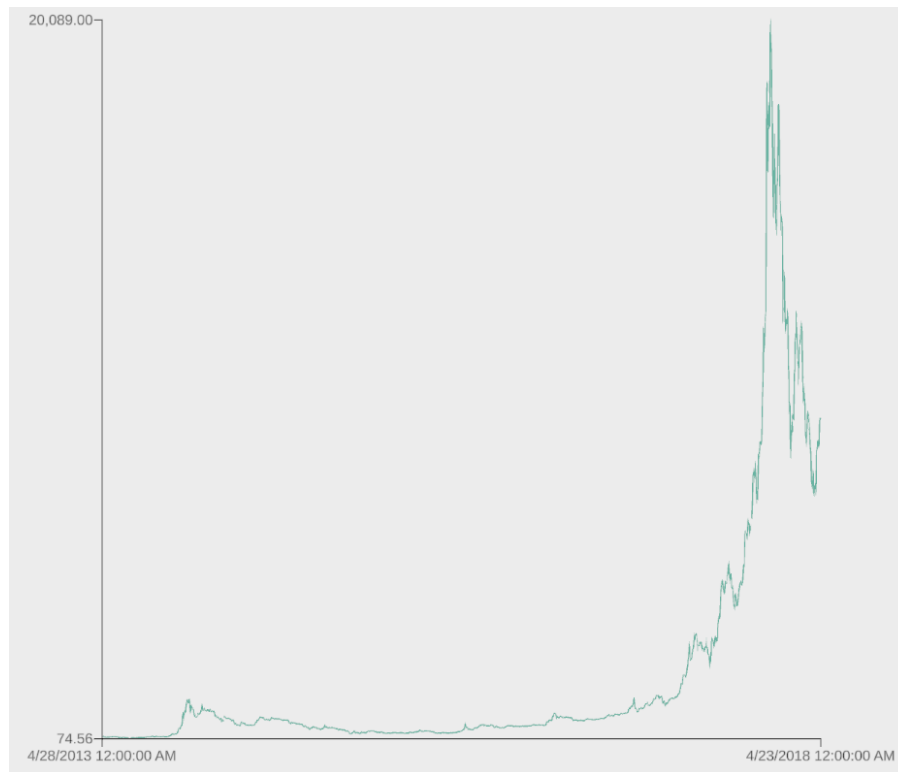
// create/update X scale and Axis
m_x_scale = new ScaleTimeFloat()
    .InputRange(min_date, max_date)
    .OutputRange(0, m_width);

m_x_axis = new AxisBottom<DateTime>(m_x_scale)
    .SetTickCount(2)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .Attach(gameObject);

// create/update Y scale and Axis
m_y_scale = new ScaleLinearFloatFloat()
    .InputRange(min_val, max_val)
    .OutputRange(0, m_height);

m_y_axis = new AxisLeft<float>(m_y_scale)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .Attach(gameObject);

// create/update the line
m_line = new Line2D<List<Record>, Record>()
    .Datum(data)
    .X(d => m_x_scale.F(d.Date))
    .Y(d => m_y_scale.F(d.Value))
    .SetMaterial(LineMaterial)
    .SetColor(new Color(105 / 255.0f, 179 / 255.0f, 162 / 255.0f))
    .Attach(gameObject);
```



```

// compute the extent of relevant attribute(s)
float min_val = 0;
float max_val = 1000;
// Utils.Extent(data, d => d.Value, out min_val, out max_val);

// bin the data
var bin_generator = new Bin<List<BarChartRecord>, BarChartRecord>()
    .Datum(data)
    .Domain(min_val, max_val)
    .Threshold(20)
    .Value(d => d.Value)
    .Generate(out Bin[] bins);

// compute the extent of the bins
Utils.Extent(bins, d => d.count, out float min_bin, out float max_bin);

// create/update X scale
m_x_scale = new ScaleLinearFloatFloat()
    .InputRange(min_val, max_val)
    .OutputRange(0, m_width);

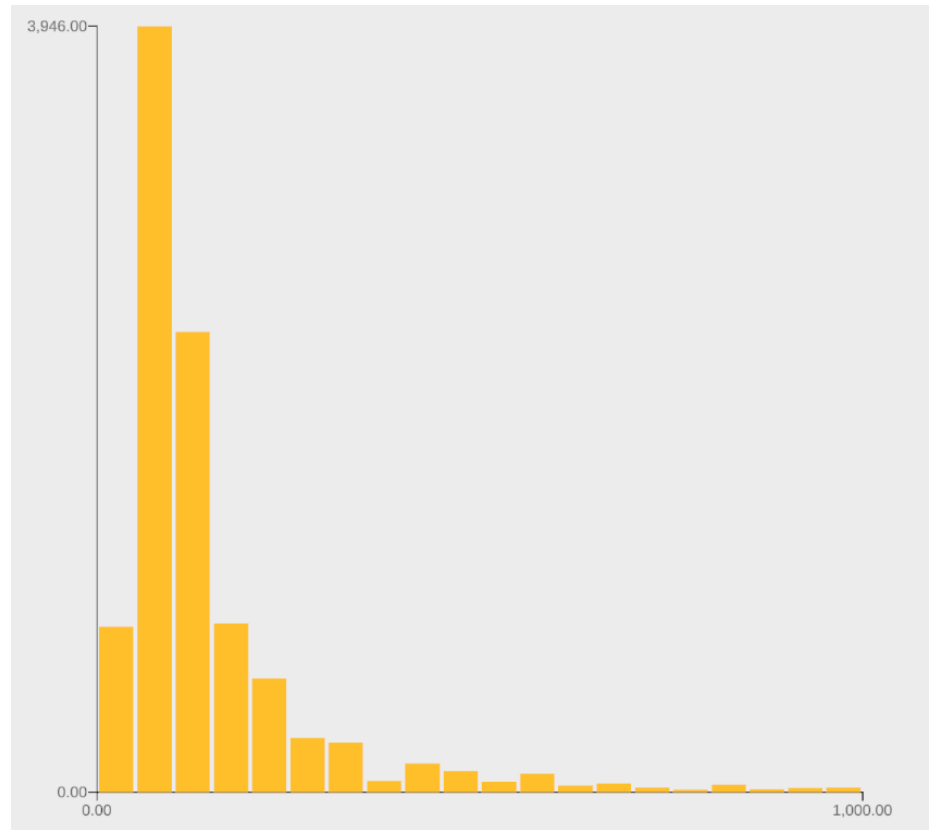
// create/update Y scale
m_y_scale = new ScaleLinearFloatFloat()
    .InputRange(0, max_bin)
    .OutputRange(0, m_height);

m_x_axis = new AxisBottom<float>(m_x_scale)
    .SetTickCount(2)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .Attach(gameObject);

m_y_axis = new AxisLeft<float>(m_y_scale)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .Attach(gameObject);

var rects = new Primitive2D<Bin[], Bin>()
    .Datum(bins)
    .Join(PrimitiveShape2D.RECT)
    .X(d => m_x_scale.F(d.x))
    .Y(d => m_y_scale.F(d.count) / 2.0f)
    .Width(d => m_x_scale.F(d.interval) - m_bin_padding)
    .Height(d => m_y_scale.F(d.count))
    .SetMaterial(RectMaterial)
    .Attach(gameObject);

```



```

// compute the extent of relevant attribute(s)
Utils.Extent(data, d => d.v0, out float min_val_x, out float max_val_x);
Utils.Extent(data, d => d.v1, out float min_val_y, out float max_val_y);

// bin the data
var bin_generator = new Bin2D<List<Record>, Record>()
    .Datum(data)
    .DomainX(min_val_x, max_val_x)
    .DomainY(min_val_y, max_val_y)
    .NbrBinsX(10)
    .NbrBinsY(10)
    .X(d => d.v0)
    .Y(d => d.v1)
    .Generate(out Bin2D[] bins);

// compute the extent of the bins
Utils.Extent(bins, d => d.count, out float min_bin, out float max_bin);

// create/update X scale
m_x_scale = new ScaleLinearFloatFloat()
    .InputRange(0, max_val_x)
    .OutputRange(0, m_width);

// create/update Y scale
m_y_scale = new ScaleLinearIntFloat()
    .InputRange(0, Mathf.RoundToInt(max_bin))
    .OutputRange(0, m_height);

// create/update Z scale
m_z_scale = new ScaleLinearFloatFloat()
    .InputRange(0, max_val_y)
    .OutputRange(0, m_depth);

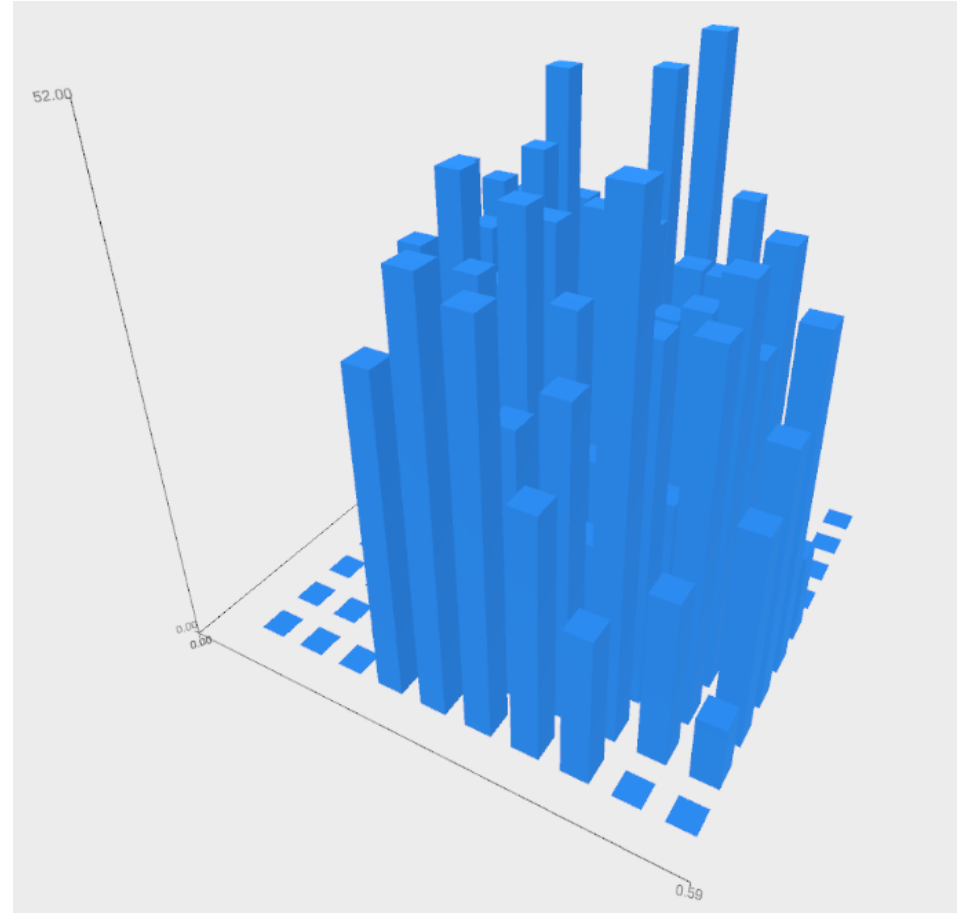
// create/update X axis
m_x_axis = new AxisBottom<float>(m_x_scale)
    .SetTickCount(2)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .SetTickLookAtCamera(true)
    .Attach(gameObject);

// create/update Y axis
m_y_axis = new AxisLeft<int>(m_y_scale)
    .SetTickCount(2)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .SetTickLookAtCamera(true)
    .Attach(gameObject);

// create/update Z axis
m_z_axis = new AxisBottom<float>(m_z_scale)
    .SetTickCount(2)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .SetTickLookAtCamera(true)
    .RotateAround(-90.0f)
    .Attach(gameObject);

var rects = new Primitive3D<Bin2D[], Bin2D>()
    .Datum(bins)
    .Join(PrimitiveShape3D.CUBE)
    .X(d => m_x_scale.F(d.x))
    .Y(d => m_y_scale.F(d.count) / 2.0f)
    .Z(d => m_z_scale.F(d.y))
    .Width(d => m_x_scale.F(d.Interval_x) - m_bin_padding)
    .Depth(d => m_z_scale.F(d.Interval_y) - m_bin_padding)
    .Height(d => Mathf.Max(m_y_scale.F(d.count), m_min_height))
    .SetMaterial(CubeMaterial)
    .Attach(gameObject);

```



```

// compute the extent of relevant attribute(s)
Utils.Extent(data, d => d.x, out float min_val_x, out float max_val_x);
Utils.Extent(data, d => d.y, out float min_val_y, out float max_val_y);
Utils.Extent(data, d => d.z, out float min_val_z, out float max_val_z);

// create/update X scale
m_x_scale = new ScaleLinearFloatFloat()
    .InputRange(min_val_x, max_val_x)
    .OutputRange(0, m_width);

// create/update Y scale
m_y_scale = new ScaleLinearFloatFloat()
    .InputRange(min_val_z, max_val_z)
    .OutputRange(0, m_height);

// create/update Z scale
m_z_scale = new ScaleLinearFloatFloat()
    .InputRange(min_val_y, max_val_y)
    .OutputRange(0, m_depth);

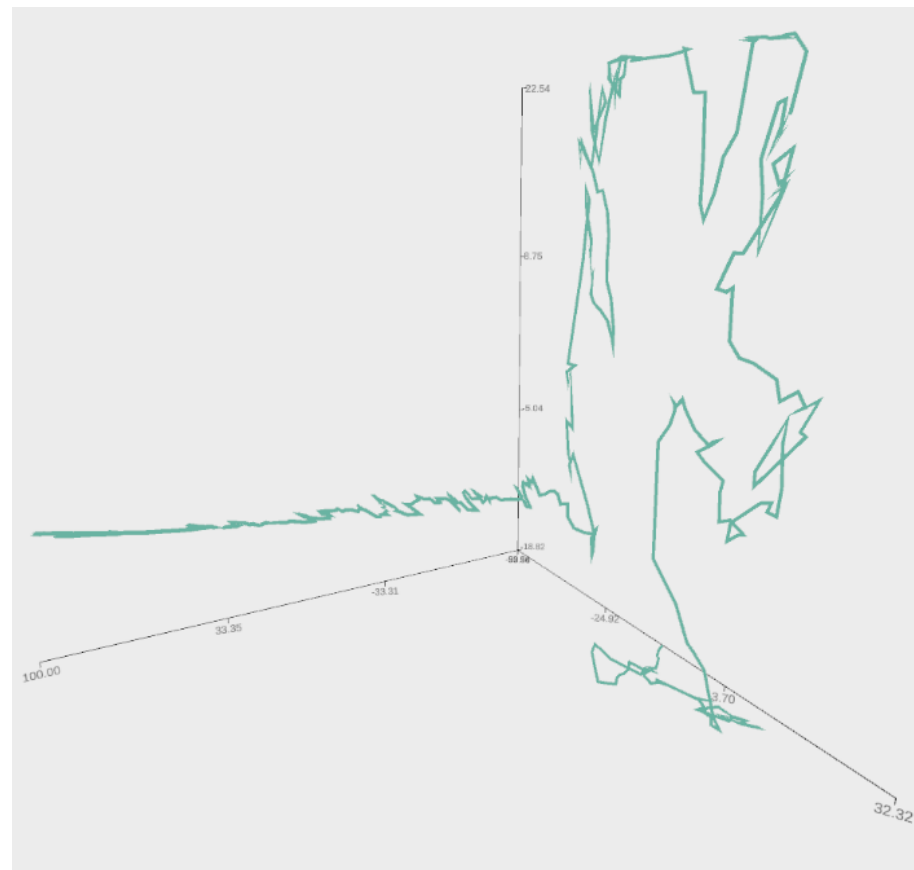
// create/update X axis
m_x_axis = new AxisBottom<float>(m_x_scale)
    .SetTickCount(4)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .SetTickLookAtCamera(true)
    .Attach(gameObject);

// create/update Y axis
m_y_axis = new AxisLeft<float>(m_y_scale)
    .SetTickCount(4)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .SetTickLookAtCamera(true)
    .Attach(gameObject);

// create/update Z axis
m_z_axis = new AxisBottom<float>(m_z_scale)
    .SetTickCount(4)
    .SetAxisMaterial(AxisMaterial)
    .SetTickMaterial(TickMaterial)
    .SetTickTextColor(Color.black)
    .SetTickLookAtCamera(true)
    .RotateAroundY(-90.0f)
    .Attach(gameObject);

// create/update the line
m_line = new Line3D<List<Record>, Record>()
    .Datum(data)
    .X(d => m_x_scale.F(d.x))
    .Y(d => m_y_scale.F(d.y))
    .Z(d => m_z_scale.F(d.z))
    .SetMaterial(LineMaterial)
    .SetColor(new Color(105 / 255.0f, 179 / 255.0f, 162 / 255.0f))
    .SetStrokeWidth(0.05f)
    .Attach(gameObject);

```



- Performs well in the range of 1 000s of objects but performance deteriorates when reaching 10 000s
- Initial target of handling “large datasets” was dropped because Geometry Shaders are not supported on mobile devices
- Similar to D3.js, our toolkit provides:
 - Scales
 - Axes
 - Line generators
 - Primitive generators
- Additionally, we add 3D support by providing:
 - 3D Primitives
 - 3D line generators

DEMO

- (these features were uploaded after the submission deadline)
- Zoom-in and panning interaction similar to that of D3.js
- Reverse mapping that allows features such as:
 - Tooltips
 - Hover-on-line
- Additional scales:
 - Radius scale
 - Color scale
- Simple instantiation of GameObjects over AR marks
- Looking forward to occasionally update this at:
<https://github.com/walcht/UXRVT>